

Experiment No. 8

Aim

Study of Pulse Width Modulation (PWM).

Apparatus

Function generator, DSO, CRO, Arduino UNO, Connecting probes and wires.

Components Required

Resistor, LED, Breadboard, Potentiometer.

Theory

Arduino UNO:

The Arduino Uno is a popular open-source microcontroller board based on the ATmega328P. The ATmega328 is a single-chip microcontroller created by Atmel in the megaAVR family. Arduino is widely used in electronics projects, embedded systems, and IoT applications due to its simplicity, affordability, and ease of use. Let us take a look at the Arduino board as seen in Fig. 1.

- **Microcontroller:** The microcontroller used on the Arduino board is essentially used for controlling all major operations. The microcontroller is used to coordinate the input taken and execute the code written in a high-level language.
- **Analog Reference pin:** Analog pins are used for general purposes like supporting 10-bit analog-to-digital conversion (ADC) which is performed using analog the Read() function. Analog pins are particularly helpful since they can store 0-255 bits which is not possible using digital pins.
- **Digital Pins:** Digital pins are used for general purposes like taking input or generating output. The commands that are used for setting the modes of the pins are pinMode(), digitalRead(), and digitalWrite() commands.

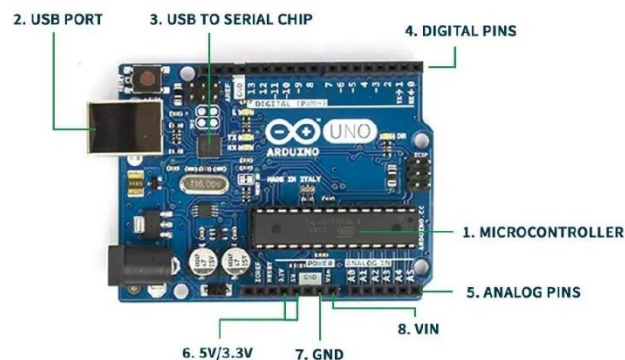


Figure 1: Arduino UNO

- **Reset Button:** The reset button on the Arduino board is used for setting all the components of Arduino to their default values.

- **Power and Ground Pins:** As the name suggests, power and ground pins are used to supply the power needed for driving the Arduino board.
- **USB (universal serial bus):** The Arduino needs certain protocols for communication purposes and the universal serial bus is used for this purpose. It helps to connect Arduino, microcontrollers with other raspberry pies.

Listing 1: PWM code

```
int ledPin = 9; // LED connected to digital pin 9
int analogPin = 3; // potentiometer connected to analog pin 3
int val = 0; // variable to store the read value

void setup() {
  // put your setup code here, to run once:
  pinMode(ledPin, OUTPUT); // sets the pin as output
}

void loop() {
  // put your main code here, to run repeatedly:

  val = analogRead ( analogPin ); // read the input pin
  analogWrite(ledPin, (val / 4)); // analogRead values go from 0
    to 1023,
    // analogWrite values from 0 to 255
}
```

Pulse Width Modulation:

Pulse Width Modulation (PWM) is a technique that controls the average power or amplitude of a signal by varying the width of pulses in a fixed period as seen in Fig. 2. Digital control is used to create a square wave, a signal switched between on and off. Alternatively, this can be seen as an analog message signal (in this case the control signal) is used to modulate the width (duty cycle) of the digital carrier (square wave). This on-off pattern can simulate voltages in between the full Vcc of the board (5V) and off (0V) by changing the portion of the time the signal spends on versus the time that the signal spends off. The duration of “on time” is called the pulse width. To get varying analog values, you change, or modulate, that pulse width. If you repeat this on-off pattern fast enough with an LED for example, the result is as if the signal is a steady voltage between 0 and Vcc controlling the brightness of the LED.

In Arduino UNO model the PWM output can be accessed on pins 3, 5, 6, 9, 10 and 11 and the pulse frequency is 980 hertz for pins 5 and 6 and 490 hertz for the other pins. The analog input of the Arduino board consists of 6 inputs and these inputs have a converter called ADC which is responsible for transforming the analog signal into a digital value that will be used by the code mentioned above. This conversion depends on the resolution of the ADC converter of each Arduino model. The Arduino UNO has a 10 bit ADC converter which means that the analog input signal that varies between 0 to 5V will be converted internally to a value between 0 and 1023 which means that if the analog input has 0V on it, the Arduino will convert this voltage internally to a digital value 0 and if the analog input has 5V on it, the Arduino will convert it to a digital value of 1023. This count value must be scaled down by 4, so that it ranges from 0-255 since it will be used in analogWrite function, as explained in next paragraph.

In the graphic below, the green lines represent a regular time period. This duration or period is the inverse of the PWM frequency. In other words, with Arduino’s PWM frequency at about 500Hz, the green lines would measure 2 milliseconds each. A call to analogWrite() is on a scale of 0 - 255, such that analogWrite(255) requests a 100% duty cycle (always on), and analogWrite(127) is a 50% duty cycle (on half the time) for example.

Potentiometer

A potentiometer is a three-terminal resistive device used to measure and adjust electrical voltage. It acts as a variable resistor or voltage divider, allowing the user to control the output voltage by adjusting its wiper position. In this device, the two outer terminals are connected to the ends of the resistive element. One middle terminal connected to the wiper, which provides a variable voltage output.

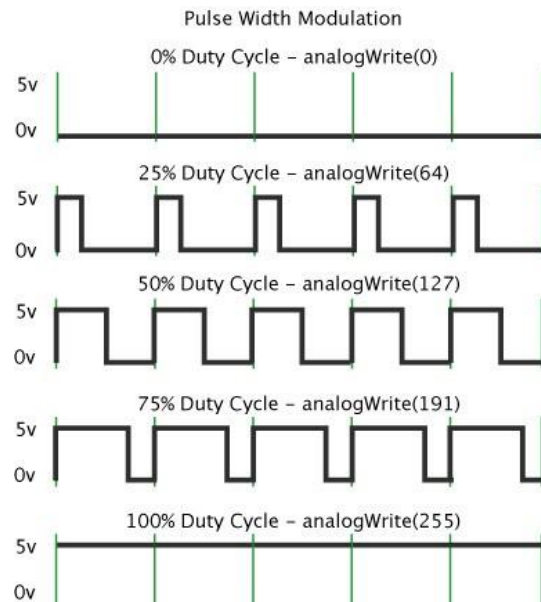


Figure 2: Pulse Width Modulation

Circuit Diagram

PWM using Potentiometer:

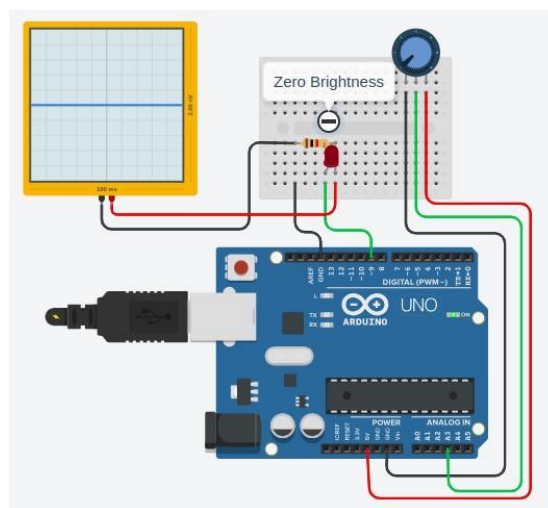


Figure 4: PWM with 0% duty cycle

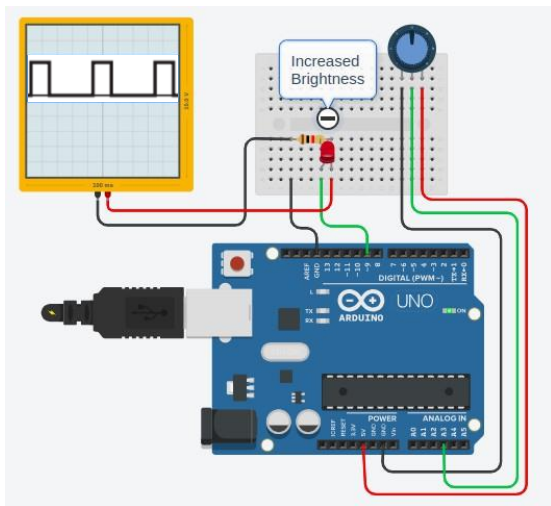


Figure 5: PWM with 50% duty cycle

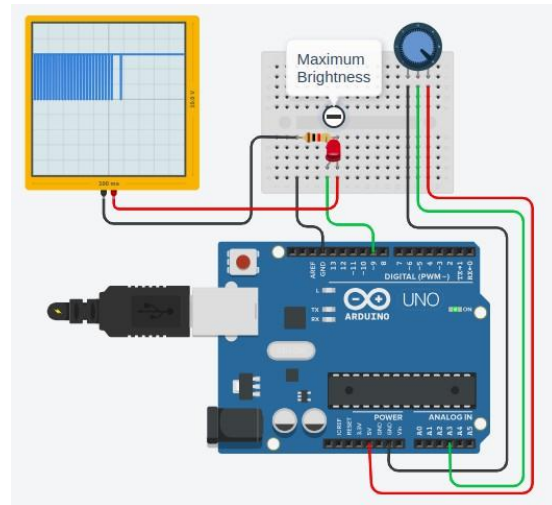


Figure 6: PWM with 100% duty cycle

Procedure

1. Step 1: Circuit Setup

- Connect the anode (+) of the LED to Arduino pin 9 (PWM pin).
- Connect the cathode (-) of the LED to one end of a 220 resistor.
- Connect the other end of the resistor to ground (GND) on the Arduino.
- Connect the function generator to an oscilloscope to monitor the PWM signal.

2. Step 2: Arduino PWM Code

- Open the Arduino IDE and write the following code:
- Upload the code to the Arduino board.
- Observe the LED gradually increasing and decreasing in brightness.

3. Step 3: Generating PWM Signal

(a) Using Potentiometer

- Upload the code to the Arduino using the IDE window. Check for any errors while compiling the code.
- Connect the circuit as shown in Fig. 3. Vary the position of the potentiometer pin to vary the output of its pin from 0 V to 5V (this is read in pin A3 of the Arduino). Observe the change in LED brightness. We are now using continuously varying voltage as modulating signal.

- iii. Connect the signal across the LED to channel 1 of the DSO. Observe the waveform on the DSO. Comment on the frequency of the square waveform.
- iv. Rotate the potentiometer pin counterclockwise until it stops and measure the voltage (V_m) on its center pin using DMM. Note it down, Observe the output on the DSO and brightness on LED. Comment.
- v. Rotate the potentiometer pin clockwise until it stops and measure the voltage (V_m) on its center pin using DMM. Note it down, Observe the output on the DSO and brightness on LED. Comment.
- vi. Rotate the potentiometer pin counterclockwise and set it some- where in between compared to 4) and 5). Measure the voltage (V_m) on its center pin using DMM. Note it down, Observe the output on the DSO and brightness on LED. Comment.

(b) Using Function generator

- i. Set the function generator to produce a sinewave from range 0 to 5V. Set V_{pp} to vary from -2.5V to +2.5V and give an offset of 2.5 V. (e.g., 50Hz sine wave).
- ii. Connect pin A3 to positive pin of function generator coaxial out- put cable and connect its ground pin to GND on Arduino. Please note, instead of connecting potentiometer center pin output to pin A3 of the Arduino, we are now using continuously varying signal as modulating signal
- iii. Connect channel 1 of the DSO across the output signal driving the LED (should already be connected from 3(a)). Connect Channel 2 of the DSO to display the message signal.
- iv. Align the positions of the message signal and PWM output as measured across LED pin on the DSO. Observe the signals and take a snapshot for report.
- v. Vary the frequency of the modulated signal from 50 to 10 Hz, observe output and take snapshots.
- vi. Vary the frequency of the modulated signal from 50 to 90 Hz, observe output and take snapshots.

Observation Table

Duty Cycle (%)	$V_m(V)$	Comments

where V_m =voltage on the center pin of pot (going into A3 of Arduino)

Conclusion

This experiment demonstrates how to achieve Pulse width modulation using a square wave digital (carrier) signal. The modulated output signal changes the duty cycle of the modulated signal according to the analog input message (or control signal). Arduino's analog output that internally applies PWM technique to control duty cycle has been utilized. This type of modulation is often used to control motor speed etc.